

Methodology: Independent Per-Asset FX Direction Prediction

with a Risk-Parity Portfolio Overlay

Architecture, Mathematical Formulation, and Implementation Reference

Repository: `mlforecasting`

Modules: `models/portfolio_lstm.py`, `models/portfolio_pnl.py`, `api/server.py`

August 3, 2026

Abstract

This document describes, in full mathematical and implementation detail, an FX direction-prediction and portfolio-construction system. N **fully independent** per-asset LSTMs (`AssetLSTM`, one per FX pair, no shared weights, no shared optimizer), each followed by a **causal self-attention** layer over the lookback window, map a rolling window of cross-sectional log-return-derived features to a heteroscedastic (μ, σ) pair at every day in the window. Training minimizes a Gaussian negative log-likelihood plus a direction-matching binary cross-entropy term, per asset, with no gradient ever crossing between assets unless an optional `cross_pairs` link widens what one asset's network *reads* (never what *optimizes* it). A calibrated directional probability is recovered via the probit link, and a configurable *neutral band* lets the model abstain on samples with insufficient edge. Downstream, an evaluation-only **risk-parity portfolio overlay** (`models/portfolio_pnl.py`) converts each asset's probability into a signal, weights it by an equal-risk-contribution solve over a rolling covariance estimate, smooths it over the model's own forecast horizon, and scales the book to a target annualized volatility – reported alongside an unmodulated risk-parity baseline. An **optional complementary training objective** lets this same portfolio Sharpe ratio feed back into training itself, in a second, genuinely joint phase layered on top of the otherwise-independent per-asset optimization. Every architectural choice is presented alongside its rationale, the governing mathematical expression, the exact source file and line range implementing it, and a worked numerical example. The document closes with a full results section training three progressively richer model configurations on three FX pairs chosen for cross-sectional diversification (EUR, JPY, and AUD legs against USD), a complete configuration reference, and a summary of every architectural decision and its motivation.

Contents

1	Notation and Conventions	3
2	System Overview	3
3	Data Pipeline and Feature Engineering	4
3.1	Price loading and log returns	4
3.2	The feature catalog	5
3.3	Standardization	7
4	Sequence Construction and Labels	7
4.1	The forward z-score label	7
4.2	Dense, per-day supervision within each window	8
4.3	Chronological split, and purge/embargo at each boundary	8

5	AssetLSTM: One Asset’s Independent Predictor	9
5.1	Causal self-attention	10
5.2	The heteroscedastic head	10
6	PredictionModel: N Assets as One Persistable Unit	10
6.1	cross_pairs: opt-in, per-pair input widening	10
7	Loss Functions	11
7.1	Gaussian negative log-likelihood	11
7.2	The direction anti-collapse term	11
7.3	The probit link	12
8	Training: Fully Independent per Asset	12
8.1	Per-asset checkpoint selection	13
8.2	Multi-seed restarts, and sweeping <code>bce_weight</code>	13
8.3	Numerical safety: immediate failure on a non-finite gradient	13
9	Calibration	14
10	The Neutral Band and Confusion-Matrix Metrics	14
11	Portfolio Construction: A Risk-Parity Overlay	14
11.1	Equal-risk-contribution (risk parity) weights	14
11.2	Probability modulation	15
11.3	Horizon smoothing: why the product, not the signal, gets averaged	15
11.4	Target-volatility scaling	16
12	Optional Complementary Objective: Training on Portfolio Sharpe	16
12.1	Why this cannot be per-asset	16
12.2	The joint training phase	17
12.3	Validation selection uses the same composite objective	18
12.4	Empirical verification of the cross-asset coupling	18
13	Results	18
13.1	Asset selection	18
13.2	Model modes	19
13.3	Directional hit rate	19
13.4	Confusion matrix (mode C, test split)	20
13.5	Portfolio Sharpe: modulated vs. risk-parity baseline	20
13.6	Colored cumulative return and predicted probability (mode C, test, EURUSD)	21
13.7	Honest limitations of this results section	22
14	Configuration Reference	23
15	Summary of Architectural Decisions	23

1 Notation and Conventions

Symbol	Code name	Meaning
N	<code>n_assets</code>	Number of tradable FX pairs
L	<code>lookback</code>	Sequence length: trailing days in one input window
C	<code>n_channels</code>	Feature channels PER PAIR (depends on the selected feature set, §3.2)
H	<code>hidden_size</code>	Each <code>AssetLSTM</code> ’s LSTM hidden state width
B	(implicit)	Number of decision samples in a split (full-batch training, no minibatching)
K	<code>n_attn_heads</code>	Attention heads in each asset’s causal self-attention layer
H_d	<code>direction_horizon</code>	Forward days the z-score label looks (default 5)
W_r	<code>rolling_stats_window</code>	Trailing window for rolling vol/skew/kurtosis features + label normalization
W_c	<code>cov_window</code>	Trailing window for the portfolio overlay’s rolling covariance (default 60)

A raw daily log return is $r_t \in \mathbb{R}^N$. The full asset-major feature block for one decision sample is $X_i \in \mathbb{R}^{L \times NC}$ (§3.2); a full split’s worth is a rank-3 tensor $X \in \mathbb{R}^{B \times L \times NC}$. Superscript “raw” denotes a pre-standardization quantity; a hat ($\hat{\cdot}$) denotes a model estimate. All neural network code is PyTorch; every `nn.LSTM` is instantiated with `batch_first=True`. Training is **full-batch**: one epoch is one forward/backward pass over the entire training split at once, never a shuffled minibatch loop – this is not merely an implementation detail but a load-bearing assumption for the optional Sharpe training objective in §12, which needs its samples in chronological order.

Every feature/behavior described below that is **off by default** is marked with an **Optional component** box; Section 14 tabulates every configuration key and its default.

2 System Overview

The system has two structurally different stages that are trained **never jointly by default**: (1) `PredictionModel`, N independent `AssetLSTM` networks, each predicting its own asset’s forward direction; and (2) an evaluation-time (or, optionally, also training-time – §12) **risk-parity portfolio overlay** that turns those N independent probabilities into a single book of position sizes.

- Stage 1. Data & features** (§3): daily close prices $\rightarrow$ log returns $\rightarrow$ a configurable per-pair feature set (raw return, rolling vol/skew/kurtosis, FX carry, moving-average crossover, causal Butterworth band-pass) $\rightarrow$ rolling windows $\rightarrow$ chronological train/validation/test split with purge/embargo $\rightarrow$ train-only standardization.
- Stage 2. AssetLSTM** (§5): N independent LSTM + causal self-attention networks, one per pair, each reading the FULL cross-sectional feature block (or a configurable subset of other pairs’ blocks via `cross_pairs`) and outputting a dense, per-day heteroscedastic (μ, σ) pair.
- Stage 3. Training** (§8): each asset’s own `torch.optim.Adam` instance minimizes Gaussian NLL + a direction BCE term, computed from *only that asset’s own* output and label – fully independent, verified empirically (§12.4). An optional second, genuinely *joint* phase (§12) can add a portfolio-Sharpe objective on top.
- Stage 4. Calibration & reporting** (§9–10): a validation-fit residual-scale correction $\hat{\sigma}_{\text{hat}}$, the probit link to a calibrated probability, a neutral band for abstention, and the standard confusion-matrix metrics.

Stage 5. Portfolio overlay (§11): each asset’s probability becomes a signal in $[-1, 1]$, weighted by an equal-risk-contribution solve, smoothed over the model’s own forecast horizon, and scaled to a target annualized volatility – reported against an unmodulated risk-parity baseline.

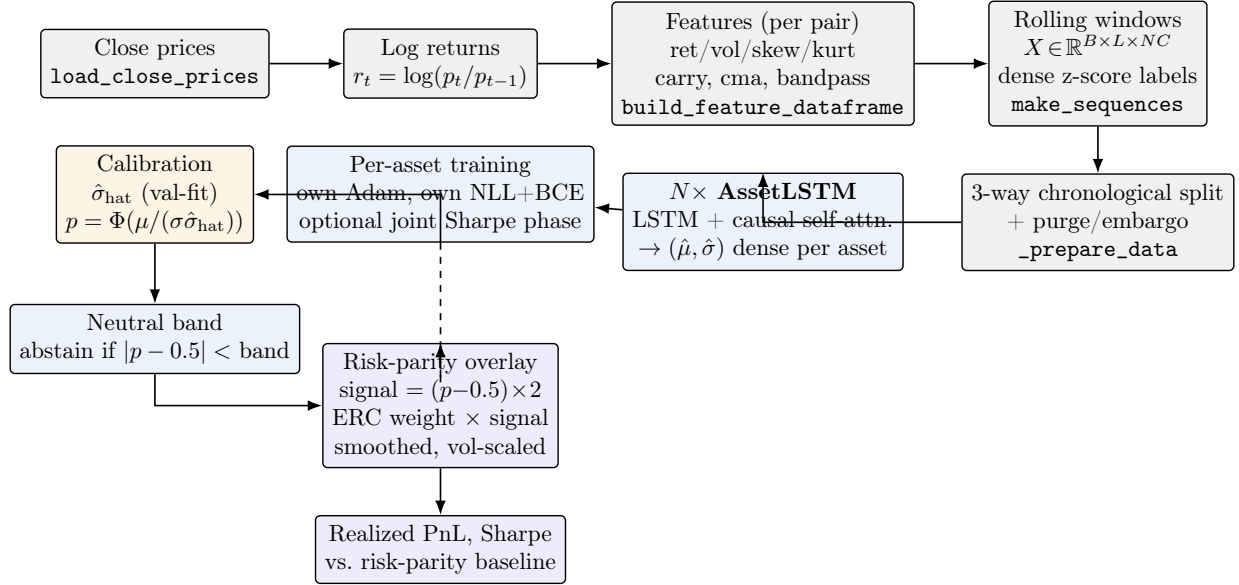


Figure 1: End-to-end pipeline. Dashed arrow: the optional joint Sharpe training phase (§12) reuses the SAME risk-parity overlay machinery inside training itself, not just at evaluation time. Grey = data/feature engineering, blue = per-asset prediction, amber = calibration/reporting, violet = portfolio overlay.

Rationale. Per-asset independence is the load-bearing design decision of the whole prediction stage: each asset’s own gradient magnitude, Adam’s per-parameter adaptive state, and which epoch its own checkpoint gets restored from depend on nothing but that asset’s own loss curve, regardless of how many other assets are being trained alongside it. This was verified *empirically*, not just by inspection: `smoke_test.py` trains an asset "A" solo versus alongside an unrelated pure-noise asset "B" (same seed, `dropout=0`) and asserts byte-identical hit rates (§12.4) – observed result: 0.485 versus 0.485, exactly. The portfolio overlay is the one place cross-asset information is genuinely combined, and even there, by default, it is pure *postprocessing*: it never touches the weights that produced the probabilities it consumes, unless the optional Sharpe objective (§12) is explicitly turned on – and even then, only that one additional phase becomes joint; the primary NLL+BCE phase remains untouched.

3 Data Pipeline and Feature Engineering

3.1 Price loading and log returns

```

1 def load_close_prices(symbols: list[str], years: int) -> pd.DataFrame:
2     end = date.today()
3     start = end - timedelta(days=365 * years)
4     prices = get_time_series(symbols, start, end)
5     missing = [s for s in symbols if s not in prices.columns or prices[s].dropna().empty]
6     if missing:
7         downloader = FXDownloader(years=years)
8         fresh = {s: downloader.download_pair(s, MAJOR_FX_PAIRS.get(s, f"{s}=X")) for s in missing}
9         upsert_pairs(fresh)
10    prices = get_time_series(symbols, start, end)
11    return prices.dropna() # inner-join on date
12

```

```

13 def to_log_returns(prices: pd.DataFrame) -> pd.DataFrame:
14     return np.log(prices / prices.shift(1)).dropna()

```

Listing 1: Close prices from Postgres, downloading/upserting on demand.

models/portfolio_lstm.py:197-229

$$r_t = \log\left(\frac{p_t}{p_{t-1}}\right) \in \mathbb{R}^N \quad (1)$$

Rationale. Log returns are additive across time ($\sum_t r_t = \log(p_T/p_0)$), which every downstream variance/covariance computation (the rolling moments in §3.2, the portfolio overlay’s covariance in §11.1) implicitly assumes. Prices are inner-joined (`dropna()`) across every requested pair so every downstream window is defined on exactly the same calendar days for every asset – a single stale/missing quote for one pair would otherwise silently misalign that day’s cross-sectional feature block.

3.2 The feature catalog

Every per-pair input channel this system knows how to build is enumerated in a single, fixed-order catalog:

```

1 FEATURE_CATALOG: tuple[str, ...] = ("log_return", "vol", "skew", "kurt", "carry", "cma", "
   bandpass")
2 DEFAULT_FEATURES: list[str] = ["log_return", "vol", "skew", "kurt"]
3 DEFAULT_CMA_WINDOWS: list[tuple[int, int]] = [[10, 50]]
4 DEFAULT_BANDPASS_WINDOWS: list[tuple[int, int]] = [[10, 50]]
5 DEFAULT_BANDPASS_ORDER: int = 3

```

Listing 2: The fixed feature catalog and its defaults.

models/portfolio_lstm.py:301-327

A caller’s **features** list is a *selection* out of this catalog, never an ordering directive – columns are always assembled in the catalog’s own fixed order, per pair, asset-major (all of one pair’s channels contiguous, then the next pair’s):

$$X_i = \left[\underbrace{r^{(1)}, \text{vol}^{(1)}, \dots}_{\text{pair 1, } C \text{ channels}}, \underbrace{r^{(2)}, \dots}_{\text{pair 2}}, \dots \right] \in \mathbb{R}^{L \times NC} \quad (2)$$

"cma" and "bandpass" are special: each contributes *one channel per window pair* in `cma_windows`/`bandpass_windows` not one channel total:

$$C = \underbrace{|\{f \in \text{features} : f \notin \{\text{cma}, \text{bandpass}\}\}|}_{\text{base features}} + \underbrace{|\text{cma_windows}| \cdot \mathbb{I}[\text{cma} \in \text{features}]}_{\text{cma channels}} + \underbrace{|\text{bandpass_windows}| \cdot \mathbb{I}[\text{bandpass} \in \text{features}]}_{\text{bandpass channels}} \quad (3)$$

models/portfolio_lstm.py:330-346 (`n_channels_per_pair`)

(a) **Raw log return.** Channel 0 whenever "log_return" is selected: r_t itself, Equation 1.

(b/c/d) **Rolling moments: volatility, skewness, excess kurtosis.**

$$\text{vol}_t = \text{std}(r_{t-W_r+1}, \dots, r_t), \quad \text{skew}_t = \text{skew}(r_{t-W_r+1}, \dots, r_t), \quad \text{kurt}_t = \text{kurt}(r_{t-W_r+1}, \dots, r_t) - 3 \quad (4)$$

(pandas’ `.kurt()` already subtracts 3, i.e. *excess* kurtosis, so a Gaussian series reports ≈ 0). $W_r = \text{rolling_stats_window}$ (default 20) is the SAME window that normalizes the forward z-score label (§4), so the label’s scale matches what the model can itself observe about recent volatility.

models/portfolio_lstm.py:276-298 (`rolling_moment_features`)

(e) Carry (interest-rate differential).

$$\text{carry}_t^{(b,q)} = \frac{\rho_t^{(b)} - \rho_t^{(q)}}{100} \quad (5)$$

for a pair with base currency b and quote currency q (e.g. EURUSD $\Rightarrow b=\text{EUR}, q=\text{USD}$), where $\rho_t^{(c)}$ is currency c 's short-term policy rate (FRED, monthly) at time t . Reindexed onto the daily index with **forward-fill only** – never back-filled, so no future rate print leaks backward; any day before the first available print is 0.

```
1 diff = (rates[base] - rates[quote]) / 100.0
2 carry[pair] = diff.reindex(carry.index, method="ffill").fillna(0.0)
```

Listing 3: Carry: forward-fill only, never back-filled.

models/portfolio_lstm.py:239-273 (load_carry)

(f) Cross moving averages (trend). For each (short, long) window pair:

$$\text{cma}_t^{(s,l)} = \bar{r}_t^{(s)} - \bar{r}_t^{(l)}, \quad \bar{r}_t^{(w)} = \frac{1}{w} \sum_{k=0}^{w-1} r_{t-k} \quad (6)$$

positive when the recent (short-window) trend runs above the longer-run (long-window) trend – the return-space analogue of a classic price moving-average crossover. Purely trailing (`min_periods=window`, no back-fill).

models/portfolio_lstm.py:349-376 (cross_moving_averages)

(g) Causal Butterworth band-pass (faster-reacting trend). An SMA-difference (Eq. 6) is a crude, high-lag approximation of a band-pass filter. A genuine Butterworth design achieves comparable noise rejection with markedly less phase lag. For a (short_period, long_period) pair in *days*, the passband (in cycles/day, Nyquist = 0.5 for once-daily-sampled data) is

$$f_{\text{low}} = \frac{1/\text{long_period}}{0.5}, \quad f_{\text{high}} = \frac{1/\text{short_period}}{0.5}, \quad 0 < f_{\text{low}} < f_{\text{high}} < 1 \quad (7)$$

(`short_period` sets the high cutoff – the fastest cycle length that still passes; `long_period` sets the low cutoff, removing the long-run trend/DC component). The filter's transfer function $H(z) = B(z)/A(z)$ (numerator/denominator polynomials of order `order`, default 3) is obtained from a standard analog Butterworth prototype via the bilinear transform (`scipy.signal.butter`), and applied via the direct-form-II transposed difference equation

$$a_0 y_t = \sum_{k=0}^P b_k r_{t-k} - \sum_{k=1}^P a_k y_{t-k} \quad (8)$$

($P = 2 \times \text{order}$ for a band-pass filter) – i.e. `scipy.signal.lfilter`, **forward-only**.

```
1 from scipy.signal import butter, lfilter
2 nyquist = 0.5
3 low = (1.0 / long_period) / nyquist
4 high = (1.0 / short_period) / nyquist
5 b, a = butter(order, [low, high], btype="bandpass")
6 filtered = lfilter(b, a, returns[pair].to_numpy(dtype=np.float64))
7 series.iloc[:long_period] = np.nan # filter transient hasn't settled yet
```

Listing 4: Causal Butterworth band-pass - `lfilter`, deliberately NOT `filtfilt`.

models/portfolio_lstm.py:379-442 (butterworth_bandpass_features)

Rationale. `scipy.signal.filtfilt` is the *usual* way a Butterworth filter is applied (it runs the filter forward, then backward, canceling phase distortion entirely) – but the backward pass uses samples *after* the point being filtered, which is non-causal and would leak future returns into today’s feature value, exactly the look-ahead leakage the rest of this pipeline (purge/embargo, trailing-only rolling features, the causal attention mask in §5) is built to avoid. `lfilter` keeps every output strictly a function of past and current samples, at the cost of some unavoidable phase lag – a genuinely causal filter can never fully cancel it. It is still empirically faster to react than a CMA crossover for comparable noise rejection (§12.4, `smoke_test.py` test 8c: peaks 5 days after a synthetic regime shift, versus a CMA crossover’s 9 days), because a proper Butterworth design shapes its passband/stopband far more efficiently than differencing two plain moving averages.

Implementation note. Numerical stability of the IIR recursion. Equation 8 is an infinite-impulse-response recursion; for a very low `long_period` cutoff (poles close to the unit circle), the direct (`b`, `a`) transfer-function form can be numerically sensitive to coefficient rounding at high order. This was investigated directly against this project’s own real FX history (8 years, 7 major pairs) for `bandpass_windows= [[5, 25], [50, 100]]`: no NaN/Inf beyond the intentional `long_period`-row transient-masking region were observed, so the current (`b`, `a`) + `lfilter` implementation was kept as-is; a second-order-sections (`output="sos"`) form is the standard, more conservative alternative if a future, more extreme window combination were ever found to be unstable.

Assembly and NaN handling. All selected channels are assembled asset-major (Eq. 2) into one wide DataFrame, then forward-filled and any still-leading NaN (not enough trailing history for the very first rows) is set to a neutral 0.0 – **never back-filled**, which would leak a later value into an earlier row:

```
1 features_df = features_df.ffill().fillna(0.0)
2 assert features_df.shape[1] == n_channels_per_pair(features, cma_windows, bandpass_windows) *
   len(pairs)
```

Listing 5: Feature assembly - one function builds every selected channel.

`models/portfolio_lstm.py:445-502 (build_feature_dataframe)`

3.3 Standardization

Per-feature mean/std are fit **only on the training split** and reused, unchanged, for validation and test (and, when loading a saved model, restored from its checkpoint rather than refit):

$$\mu = \text{mean}_{(B,L)}(X^{\text{train,raw}}), \quad \sigma = \text{std}_{(B,L)}(X^{\text{train,raw}}) + 10^{-8}, \quad X^{\text{split}} = \frac{X^{\text{split,raw}} - \mu}{\sigma} \quad (9)$$

`models/portfolio_lstm.py:505-507 (standardize)`, 1954-1960 (`_prepare_data`)

4 Sequence Construction and Labels

4.1 The forward z-score label

For each pair, the label is the **forward, direction _ horizon-day cumulative return**, expressed in trailing-volatility standard deviations:

$$z_d = \frac{\sum_{k=1}^{H_d} r_{d+k}}{\text{vol}_d \cdot \sqrt{H_d}} \quad (10)$$

where vol_d is the SAME trailing rolling volatility used as an input feature (Eq. 4) – known as of day d ’s close, no look-ahead – and the $\sqrt{H_d}$ scaling is the standard i.i.d.-returns assumption that an H_d -day sum’s standard deviation scales as the daily standard deviation times $\sqrt{H_d}$. This

keeps each move's *magnitude*, not just its sign, and normalizes it consistently with the volatility input feature so a 1% move on a quiet pair and a 3% move on a volatile one can land at a similar z-score if both are equally "surprising" relative to their own recent volatility.

```
1 padded_cumsum = np.concatenate([np.zeros((1, n)), np.cumsum(values, axis=0)], axis=0)
2 forward_sum = padded_cumsum[H+1 : valid+H+1] - padded_cumsum[1 : valid+1]
3 z[:valid] = forward_sum / (trailing_vol[:valid] * np.sqrt(H) + eps)
```

Listing 6: Vectorized forward z-score label via a cumulative-sum trick.

models/portfolio_lstm.py:510-544 (`_forward_zscore_labels`)

Rationale. $\text{sign}(z_d)$ is exactly the realized direction – what hit rate/confusion matrices use – but the CONTINUOUS magnitude is what AssetLSTM is actually supervised against (as the mean of a Gaussian, §7): a pure classification label throws away information about how confidently a given day's move should be predicted, which a proper scoring rule like Gaussian NLL can use to calibrate σ per sample.

4.2 Dense, per-day supervision within each window

```
1 for start in range(first_start, last_start):
2     day_t = start + lookback - 1 # the decision day
3     X.append(feature_values[start : day_t + 1]) # (L, NC)
4     next_returns.append(values[day_t + 1]) # (N,) - raw, for PnL only
5     z_labels.append(full_z[start : day_t + 1]) # (L, N) - one label PER DAY in the window
6     dates.append(index[day_t + 1])
```

Listing 7: Sliding window construction - X, next-day returns, and DENSE per-day labels.

models/portfolio_lstm.py:547-623 (`make_sequences`)

Crucially, `z_labels[i]` carries **one label per day inside the window**, not just at the decision day: window sample i 's label at position k is pair p 's own H_d -day-forward z-score computed as of window-day k – entirely a function of days *after* k , so supervising every position in the window is never look-ahead leakage, merely "does everything computable at day k 's close accurately predict day k 's own forward outcome", repeated for every k . `next_returns[i]`, by contrast, is the single *raw*, un-featurized return realized the day after the window – used only for realized-PnL reporting (§11), never as a training target.

Tensor shapes. $X \in \mathbb{R}^{B \times L \times NC}$; $\text{next_returns} \in \mathbb{R}^{B \times N}$; $\text{z_labels} \in \mathbb{R}^{B \times L \times N}$ (dense).

4.3 Chronological split, and purge/embargo at each boundary

The split is strictly chronological – the most recent `test_frac` fraction of all samples is carved off *first* as test, then `train_frac` splits whatever remains:

$$n_{\text{test}} = \lfloor n_{\text{total}} \cdot \text{test_frac} \rfloor, \quad n_{\text{remaining}} = n_{\text{total}} - n_{\text{test}}, \quad n_{\text{train}} = \lfloor n_{\text{remaining}} \cdot \text{train_frac} \rfloor \quad (11)$$

Because consecutive samples are stride-1, a sample's forward label window and its lookback input window overlap heavily with its neighbors' – right at a split boundary this would let information the model was trained to predict leak into what the next split "sees" as recent history. `purge_gap` = $L + H_d$ samples are dropped *entirely* (not reassigned) at each boundary that genuinely separates two non-empty splits:

```
1 purge_gap = lookback + direction_horizon
2 train_end = max(n_train - purge_gap, 0) if n_train < n_remaining else n_train
3 val_end = max(n_remaining - purge_gap, n_train) if n_test > 0 else n_remaining
```


Listing 8: Purge/embargo - only at a boundary that actually separates two splits.

models/portfolio_lstm.py:1902-1952 (`_prepare_data`)

Implementation note. The conditional guards matter. An earlier version purged `train_end/val_end` unconditionally. This is correct whenever `train_frac < 1` (training mode: there really is a validation split after train, and a test split after that), but **evaluation mode** (the API's `/api/evaluate` endpoint) deliberately sets `train_frac=1.0`, `test_frac=0.0` so the WHOLE fetched period is one continuous evaluation block. With the unconditional version, purging would still trim `purge_gap` (≈ 35) days off the *end* of that block even though there is no downstream split left to protect from leakage – needlessly discarding the freshest, most valuable days for no reason. The guards above (`n_train < n_remaining`, `n_test > 0`) make purging conditional on a boundary actually existing.

5 AssetLSTM: One Asset's Independent Predictor

Each pair gets its own dedicated AssetLSTM – **no parameters shared with any other asset**. Its *input*, however, can be the full cross-sectional feature block (or a configurable subset – §6.1), so its recurrence can still react to another pair's same-day move without a separate cross-asset stage or the bottleneck of only seeing a compressed summary of it.

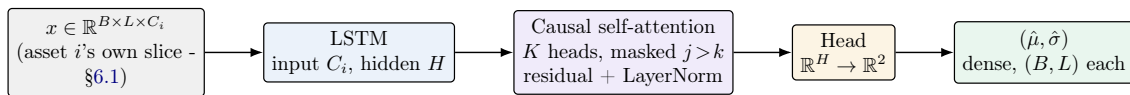


Figure 2: One AssetLSTM. Every pair gets an independently-parameterized instance of this exact block; only the input width $C_i = |\text{included_indices}_i| \times C$ can differ (§6.1).

```

1 class AssetLSTM(nn.Module):
2     def __init__(self, input_size, hidden_size=16, num_layers=1, dropout=0.0, n_attn_heads=4):
3         self.lstm = nn.LSTM(input_size, hidden_size, num_layers, batch_first=True)
4         self.attn = nn.MultiheadAttention(hidden_size, n_attn_heads, dropout=dropout,
5                                           batch_first=True)
6         self.attn_norm = nn.LayerNorm(hidden_size)
7         self.head = nn.Linear(hidden_size, 2)
8         with torch.no_grad():
9             self.head.bias[1].fill_(0.5413) # softplus(0.5413) = 1.0 -> sigma ~ 1 at init
10
11     def forward(self, x): # x: (B, L, input_size)
12         lstm_out, _ = self.lstm(x) # (B, L, H)
13         lookback = lstm_out.shape[1]
14         causal_mask = torch.triu(torch.full((lookback, lookback), float("-inf")), diagonal=1)
15         attn_out, _ = self.attn(lstm_out, lstm_out, lstm_out, attn_mask=causal_mask,
16                                need_weights=False)
17         h = self.attn_norm(lstm_out + attn_out) # residual + LayerNorm
18         out = self.head(h) # (B, L, 2)
19         mu = out[..., 0]
20         sigma = F.softplus(out[..., 1]) + 1e-3
21         return mu, sigma
  
```

Listing 9: AssetLSTM: LSTM, causal self-attention, heteroscedastic head.

models/portfolio_lstm.py:630-724

5.1 Causal self-attention

The LSTM’s hidden state at day k already summarizes $x_{0..k}$ recurrently, but that summary is a fixed-size bottleneck. Self-attention lets day k ’s representation pull directly from any *earlier* day’s hidden state:

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V, \quad M_{jk} = \begin{cases} 0 & j \leq k \\ -\infty & j > k \end{cases} \quad (12)$$

with $Q = K = V = \text{lstm_out}$ (self-attention) and M the upper-triangular causal mask (Eq. 12, `torch.triu(..., diagonal=1)`). The mask is **mandatory**: this network outputs a prediction at every day in the window, each one supervised as if computed using only information through that day (§4); an unmasked (bidirectional) attention layer would let day k ’s prediction see days after k – exactly the label look-ahead leakage the rest of this pipeline is built to avoid. Implemented as a residual block (post-norm Transformer pattern), so early in training, before the attention weights have learned anything useful, the LSTM’s own signal still passes through unchanged:

$$h = \text{LayerNorm}(\text{lstm_out} + \text{Attn}(\text{lstm_out})) \quad (13)$$

Rationale. Empirically verified, not just asserted: `smoke_test.py` test 1b perturbs only the LAST timestep of a window and confirms every EARLIER day’s prediction is byte-identical before and after (`torch.allclose(mu_a[:, :-1, :], mu_b[:, :-1, :])`) while the last day’s own prediction DOES change – direct proof the mask is doing its job, not merely present in the code.

5.2 The heteroscedastic head

$$\hat{\mu}_{i,k} \in \mathbb{R} \text{ (unbounded)}, \quad \hat{\sigma}_{i,k} = \text{softplus}(\text{out}_{i,k,1}) + 10^{-3} \text{ (strictly positive)} \quad (14)$$

The head’s sigma-half bias is initialized so `softplus(bias)  $\approx 1.0$` – the network starts out claiming predictive std = 1 (the label’s own unconditional scale), i.e. $p \approx \Phi(\hat{\mu}) \approx 0.5$ while $\hat{\mu}$ is still near its own small random init – a calibrated-ignorance starting point rather than random over/under-confidence the network must first unlearn.

Implementation note. The 10^{-3} floor matters for a different, real reason. If $\hat{\sigma}$ is driven toward 0 by a large-capacity network overfitting a single sample’s residual early in training, both `gaussian_nll` (Eq. 16) and `direction_bce` (Eq. 17) divide by σ – an unbounded-below NLL surface as $\sigma \rightarrow 0^+$. The floor bounds $1/\sigma$ at 1000; this does NOT, by itself, prevent exploding gradients on a sufficiently large/deep architecture – see the documented `device="mps" + num_layers>1` numerical issue in §8.3, discovered via exactly this failure mode.

6 PredictionModel: N Assets as One Persistable Unit

`PredictionModel` owns N `AssetLSTM` instances (`nn.ModuleList`) purely as a persistence/inference convenience – `forward()` runs every asset for batched inference, but §8 shows training never actually calls this combined `forward()`.

6.1 cross_pairs: opt-in, per-pair input widening

Optional component (opt-in, default off). `cross_pairs` (default `{}`) is a `{pair: [other_pair, ...]}` dict. DEFAULT: every pair’s own LSTM sees *only* its own C -channel block – fully independent, no cross-asset mixing at all. A pair listed as a key gets its OWN features (always included) plus every listed other pair’s FULL feature block.

```

1 pair_index = {p: i for i, p in enumerate(self.pairs)}
2 self.included_indices = []
3 for pair in self.pairs:
4     ordered = [pair] + [p for p in self.cross_pairs.get(pair, []) if p != pair]
5     indices = [pair_index[p] for p in dict.fromkeys(ordered)] # de-duplicated, own pair first
6     self.included_indices.append(indices)
7
8 self.assets = nn.ModuleList([
9     AssetLSTM(len(indices) * n_channels, hidden_size, num_layers, dropout, n_attn_heads)
10    for indices in self.included_indices
11 ])

```

Listing 10: Building each asset's own input-slice indices.

models/portfolio_lstm.py:786-822

$$C_i = |\text{included_indices}_i| \cdot C, \quad \text{asset_input}(x, i) = [x_{:, \text{idx} \cdot C : (\text{idx}+1) \cdot C}]_{\text{idx} \in \text{included_indices}_i} \quad (15)$$

Unknown pairs referenced in `cross_pairs` raise a `ValueError` immediately (construction time), rather than silently being ignored.

Rationale. This replaced an earlier design where *every* asset unconditionally saw *every* pair's features – unconditional full mixing had the same bottleneck-avoidance goal but no way to dial it back down when it wasn't paying for itself on a given pair. `cross_pairs` makes cross-asset information flow a per-pair, explicit, opt-in choice, with "fully independent" as the honest default. Verified (§12.4, smoke_test.py test 1a): by default `included_indices == [[0], [1], [2]]` for 3 pairs; linking one pair to the other two widens ONLY that pair's own input, e.g. `[[0,1,2], [1], [2]]`.

7 Loss Functions

7.1 Gaussian negative log-likelihood

$$\mathcal{L}_{\text{NLL}}(\mu, \sigma, z) = \frac{1}{BL} \sum_{i,k} \frac{1}{2} \left(2 \log \sigma_{i,k} + \left(\frac{z_{i,k} - \mu_{i,k}}{\sigma_{i,k}} \right)^2 \right) \quad (16)$$

```

1 def gaussian_nll(mu, sigma, target):
2     return (0.5 * (2.0 * torch.log(sigma) + ((target - mu) / sigma) ** 2)).mean()

```

Listing 11: Gaussian NLL - the constant $\log 2\pi$ term is dropped (doesn't affect gradients).

models/portfolio_lstm.py:1034-1045

The proper scoring rule for a heteroscedastic (μ, σ) head: unlike a plain point-estimate loss, the model can LOWER its loss on unpredictable samples by RAISING σ there (an honest "I don't know", $p \rightarrow 0.5$) while keeping σ small – and getting full credit – on samples where μ genuinely discriminates.

7.2 The direction anti-collapse term

A bare regression loss (Eq. 16 alone) is minimized by $\mu \rightarrow \mathbb{E}[z]$ – one constant sign for every sample whenever the target is barely predictable, i.e. the degenerate all-recall/zero-specificity confusion matrix. The BCE term is instead minimized by matching each SAMPLE's own sign:

$$p_{i,k} = \Phi(\mu_{i,k}/\sigma_{i,k}), \quad \mathcal{L}_{\text{BCE}} = -\frac{1}{BL} \sum_{i,k} \left[y_{i,k} \log p_{i,k} + (1 - y_{i,k}) \log(1 - p_{i,k}) \right], \quad y_{i,k} = \mathbb{1}[z_{i,k} > 0] \quad (17)$$

```

1 def direction_bce(mu, sigma, target_z):
2     score = mu / sigma
3     y = (target_z > 0).to(score.dtype)
4     log_p = torch.log(torch.clamp(torch.special.ndtr(score), min=1e-12))
5     log_not_p = torch.log(torch.clamp(torch.special.ndtr(-score), min=1e-12))
6     return -(y * log_p + (1.0 - y) * log_not_p).mean()

```

Listing 12: Direction BCE, via ndtr (NOT log_ndtr - see the note below).

models/portfolio_lstm.py:1048-1077

The total per-asset loss is $\mathcal{L} = \mathcal{L}_{\text{NLL}} + \lambda_{\text{bce}} \mathcal{L}_{\text{BCE}}$ (`bce_weight` λ_{bce} , default 1.0; 0 recovers pure distributional regression, collapse risk and all). Where nothing in the features discriminates, the BCE optimum is exactly $p = \text{base rate} \approx 0.5$, which the neutral band (§10) then reports as an honest abstention rather than a coin-flip call.

Implementation note. Why ndtr, not log_ndtr. `torch.special.log_ndtr` is numerically nicer (avoids materializing a probability that could round to exactly 0 or 1), but is **not implemented on the MPS backend** as of this codebase’s target PyTorch version – and this runs every single training epoch. Reaching for it would force a CPU-fallback environment variable on every call rather than running natively on Apple Silicon’s Metal backend. The `clamp(min=1e-12)` floor only ever binds for astronomically extreme/miscalibrated scores, far outside anything a z-score head trained under Eq. 16 actually produces. This exact substitution (`ndtr` instead of `log_ndtr`) is itself the fix for a real MPS `NotImplementedError` hit earlier in this project’s development.

7.3 The probit link

$$\Phi(z) = P(\mathcal{N}(0, 1) \leq z), \quad p = \Phi\left(\frac{\mu}{\sigma \hat{\sigma}_{\text{hat}}}\right) \quad (18)$$

`torch.special.ndtr` – the standard normal CDF, recovering a calibrated probability from a continuous z-score. $z = 0$ maps to exactly 0.5, so every downstream threshold-at-0.5 consumer (hit rate, confusion matrix) agrees exactly with `sign(z)`. models/portfolio_lstm.py:1011-1031 (probit)

8 Training: Fully Independent per Asset

Every asset gets its OWN `torch.optim.Adam` instance (own momentum/variance state, entirely separate from every other asset’s) and its OWN loss, computed from just that asset’s own (μ, σ) and label:

```

1 optimizers = [torch.optim.Adam(model.assets[i].parameters(), lr=lr, weight_decay=weight_decay)
2                 for i in range(n_assets)]
3 for epoch in range(1, epochs + 1):
4     for i in range(n_assets):
5         optimizers[i].zero_grad()
6         x_i = model.asset_input(X_train, i)
7         mu_i, sigma_i = model.assets[i](x_i) # (n_train, lookback)
8         target_i = z_labels_train[:, :, i]
9         loss_i = gaussian_nll(mu_i, sigma_i, target_i) + bce_weight * direction_bce(mu_i,
10                                         sigma_i, target_i)
11         loss_i.backward()
12         _assert_finite_grad(model.assets[i].parameters(), f"epoch {epoch}, asset {i}")
13         optimizers[i].step()

```

Listing 13: Per-asset training loop - own optimizer, own loss, own backward/step.

models/portfolio_lstm.py:1304-1479 (train_prediction_model)

Applied to the FULL dense (B, L) output for that asset – not just the window’s last day – since every day in every window is its own supervised sample (§4), and because consecutive windows slide by one day, a given calendar day is trained on repeatedly (once per window it falls inside) rather than just once, with no need to separately up-weight it.

Rationale. Nothing about how asset i trains – gradient magnitude, Adam’s per-parameter adaptive state, which epoch its checkpoint gets restored from – depends on how many OTHER assets are being trained alongside it, or how they are doing. An asset linked to another pair’s features via `cross_pairs` still trains this way: `cross_pairs` only widens what its LSTM READS, never what optimizes it. **Empirically verified** (§12.4, `smoke_test.py` test 1c): asset "A" trained solo reports hit rate 0.485; the SAME asset "A" trained alongside an unrelated pure-noise asset "B" (same seed, `dropout=0` to remove an RNG-interleaving confound) also reports 0.485 – bit-for-bit identical.

8.1 Per-asset checkpoint selection

If a validation split is tracked, the SAME per-asset loss – but on the decision day only, matching what is actually reported – is computed after every epoch, and each asset’s OWN best-epoch state dict is restored independently at the end:

```
1 if val_loss_i < best_val_loss[i]:
2     best_val_loss[i] = val_loss_i
3     best_state[i] = copy.deepcopy(model.assets[i].state_dict())
4 ...
5 for i in range(n_assets):
6     model.assets[i].load_state_dict(best_state[i]) # each asset's OWN best epoch
```

Listing 14: Per-asset validation tracking and independent checkpoint restoration.

models/portfolio_lstm.py:1413-1423, 1498-1524

Deliberately *not* selected on validation hit rate: hit rate is a coarse, saturating metric (once every sample’s sign is correct it cannot improve further no matter how much more confident/well-calibrated the predictions get), so selecting on it tends to freeze onto whichever epoch first reached its ceiling – often very early – discarding every later epoch that kept reducing loss without being able to push accuracy any higher.

8.2 Multi-seed restarts, and sweeping bce_weight

`n_seeds > 1` trains that many independent restarts and keeps whichever had the lowest validation log loss (a proper scoring rule, unlike accuracy, which saturates – same reasoning as checkpoint selection above). `bce_weight` additionally accepts a **list** to sweep alongside the seeds: every value trains under every seed (reset to the SAME initial weights per seed, isolating the value’s own effect from initialization luck), and whichever validated best wins, first per seed then overall.

models/portfolio_lstm.py:2242-2329 (`run_pipeline_multi_seed`)

8.3 Numerical safety: immediate failure on a non-finite gradient

Implementation note. A confirmed PyTorch MPS bug. A real training run with `num_layers=2`, `hidden_size=128`, `device="mps"` produced a perfectly FINITE loss (≈ 1.46) but a NaN gradient on its very first `backward()` call – bisected against `hidden_size=128` alone (finite), `num_layers=2` alone (finite), `n_attn_heads=8` alone (finite): only the `hidden_size=128` + `num_layers=2` combination together produces the NaN, and only on `device="mps"` – the identical forward/backward pass on `device="cpu"` gives a normal, finite gradient (≈ 3.53). This is a PyTorch Metal-backend multi-layer-LSTM backward-kernel bug, not a bug in this codebase’s mathematics. `_assert_finite_grad` (called right after every `backward()`, both the per-asset phase above and the optional joint Sharpe phase in §12) cannot fix PyTorch’s own kernel, but turns "N silently-wasted epochs producing a NaN

checkpoint" into an immediate, actionable error naming the exact epoch/asset and pointing at this issue. A proactive `logger.warning` additionally fires whenever `device="mps"` and `num_layers > 1` are configured together, before training even starts.

9 Calibration

The model's own per-sample σ is trained via Eq. 16, but a systematic, GLOBAL over/under-confidence can remain. A single scalar correction per asset, $\hat{\sigma}_{\text{hat}}$, is fit **once**, on the VALIDATION split ONLY (never train, which would fit the calibration to noise the model already overfit; never test, which would leak the held-out set into a quantity that ships with the model):

$$\hat{\sigma}_{\text{hat}} = \max\left(\text{std}\left(\frac{z^{\text{val}} - \mu^{\text{val}}}{\sigma^{\text{val}}}\right), 10^{-3}\right), \quad p = \Phi\left(\frac{\mu}{\sigma \cdot \hat{\sigma}_{\text{hat}}}\right) \quad (19)$$

`models/portfolio_lstm.py:2050-2062 (evaluate_prediction_model)` If the NLL-trained sigmas are already honest, $\hat{\sigma}_{\text{hat}} \approx 1$ and this is a no-op; if they are collectively over/under-confident, $\hat{\sigma}_{\text{hat}}$ corrects the shared factor. Persisted in the model's checkpoint, so live single-window inference (which has no validation set to refit against) reuses the training-time estimate.

10 The Neutral Band and Confusion-Matrix Metrics

$$\text{decision} = \begin{cases} \text{positive} & p > 0.5 + \text{band} \\ \text{negative} & p < 0.5 - \text{band} \\ \text{abstain} & \text{otherwise} \end{cases} \quad (20)$$

`models/portfolio_lstm.py:1084-1158 (apply_neutral_band, confusion_matrix_metrics)`

Every derived metric (accuracy, precision, recall, specificity, F1) is computed over DECIDED samples only; **coverage** = $n_{\text{decided}} / (n_{\text{decided}} + n_{\text{abstained}})$ reports how often the model actually makes a call – accuracy and coverage must always be read together (100% accuracy at 2% coverage is a very different claim from 55% at 80%).

Implementation note. Pure postprocessing, never part of training. `train_prediction_model` never receives `neutral_band`; `evaluate_prediction_model` stores `probabilities_* RAW` (unsnapped). This means a confusion matrix / hit rate / colored cumulative-return chart for a DIFFERENT band never requires retraining or even a new evaluation pass – the web app exposes a live "Neutral band" control that recomputes all three entirely client-side (`frontend/src/metrics.js`, a numerically-verified JS port of Eq. 20) from the raw probability + realized label pairs the API ships alongside every result.

11 Portfolio Construction: A Risk-Parity Overlay

Everything in this section is **evaluation-only postprocessing** by default (never touches training, unless the optional objective in §12 is enabled) – implemented in `models/portfolio_pnl.py`, entirely separate from the prediction network.

11.1 Equal-risk-contribution (risk parity) weights

A rolling, CAUSAL sample covariance is estimated from realized returns only (never model predictions):

$$\hat{\Sigma}_t = \frac{1}{W_c - 1} \sum_{\tau=t-W_c}^{t-1} (r_\tau - \bar{r})(r_\tau - \bar{r})^\top + 10^{-10} I, \quad \bar{r} = \frac{1}{W_c} \sum_{\tau=t-W_c}^{t-1} r_\tau \quad (21)$$

using STRICTLY returns before t (never t itself), undefined (NaN) below `min_periods=20` observations. The equal-risk-contribution weight w_t solves

$$w_t = \arg \min_{w \succeq 0, \mathbf{1}^\top w = 1} \sum_{i=1}^N \left(w_i (\hat{\Sigma}_t w)_i - \frac{w^\top \hat{\Sigma}_t w}{N} \right)^2 \quad (22)$$

i.e. every asset's contribution to total portfolio variance, $w_i (\hat{\Sigma}_t w)_i$, is pushed to the same share $\frac{1}{N}$ of the total – long-only, fully invested, no closed form once assets are correlated, solved numerically per day (SLSQP).

```

1 def risk_parity_weights(cov):
2     scale = np.trace(cov) / n; cov = cov / scale # see fixnote below
3     def objective(w):
4         port_var = w @ cov @ w
5         contrib = w * (cov @ w)
6         return np.sum((contrib - port_var / n) ** 2)
7     result = minimize(objective, np.full(n, 1/n), method="SLSQP",
8                       bounds=[(1e-6, 1.0)] * n, constraints=[{"type": "eq", "fun": lambda w: w.
9                               sum() - 1}])
10    return np.clip(result.x, 1e-6, None) / np.clip(result.x, 1e-6, None).sum()

```

Listing 15: Equal-risk-contribution solve (SLSQP), rescaled for numerical conditioning.

models/portfolio_pnl.py:62-164 (rolling_covariance_matrices, risk_parity_weights, precompute_risk_parity)

Implementation note. A real bug, found and fixed. Daily FX return VARIANCES are $\sim 10^{-6}$, putting the raw objective (Eq. 22, squared) at $\sim 10^{-12}$. SLSQP's finite-difference gradient rounds to noise at that scale and the optimizer exits after ONE iteration, silently returning the untouched equal-weight starting guess – confirmed directly against a real 3×3 FX covariance matrix (weights [0.333, 0.333, 0.333], but risk contributions [1.44e-6, 2.08e-6, 9.3e-8] – visibly NOT equalized). The fix: rescale cov to $O(1)$ diagonal entries before optimizing. The ERC solution is scale-invariant ($\hat{\Sigma} \rightarrow k\hat{\Sigma}$ leaves $\arg \min w$ unchanged – both terms in Eq. 22 scale by k^2), so this changes nothing about the answer, only its numerical conditioning. Re-verified after the fix: weights [0.326, 0.239, 0.435], risk contributions equalized to 4 significant figures.

11.2 Probability modulation

Each day, the model's RAW probability $p_i(t)$ (never band-snapped – this strategy always has a view) becomes a signal in $[-1, 1]$:

$$s_i(t) = (p_i(t) - 0.5) \times 2 \quad (23)$$

and the day's raw modulated weight is the ELEMENTWISE product with the risk-parity weight, $\text{raw}_i(t) = w_i(t) s_i(t)$ – can go negative (short) when $p_i(t) < 0.5$.

11.3 Horizon smoothing: why the product, not the signal, gets averaged

The book rebalances DAILY, but $p_i(t)$ forecasts a H_d -day-forward move – consecutive days' signals are therefore forecasts of heavily overlapping windows (sharing $H_d - 1$ days), highly autocorrelated information to trade fresh every day. The modulated weight is smoothed with a trailing simple moving average over H_d days BEFORE vol-scaling:

$$\overline{\text{raw}}_i(t) = \frac{1}{\min(H_d, t + 1)} \sum_{\tau=\max(0, t-H_d+1)}^t \text{raw}_i(\tau) \quad (24)$$

models/portfolio_pnl.py:120-132 (_trailing_moving_average)

Implementation note. Order of operations matters, and was originally wrong in the training-time counterpart. Because the risk-parity weight $w_i(t)$ itself drifts day to day (a rolling covariance solve), $\text{MA}(w \cdot s) \neq w(t) \cdot \text{MA}(s)$ in general. Eq. 24 smooths the PRODUCT, matching the evaluation-time strategy exactly. An early version of the differentiable training-time counterpart (§12) smoothed the SIGNAL alone and multiplied by the *current-day* risk-parity weight afterward – a real inconsistency between what training optimized and what evaluation actually reported, caught by direct code comparison and fixed to match Eq. 24 precisely.

The unmodulated risk-parity BASELINE is not smoothed (it carries no forecast, so there is no overlap to average out) but IS scaled to the same target volatility, for a like-for-like comparison.

11.4 Target-volatility scaling

$$\sigma_{\text{book}}(t) = \sqrt{w(t)^\top \hat{\Sigma}_t w(t)}, \quad \text{scale}(t) = \min\left(\frac{\text{target_vol}/\sqrt{252}}{\max(\sigma_{\text{book}}(t), 10^{-8})}, 20\right), \quad \hat{w}(t) = w(t) \cdot \text{scale}(t) \quad (25)$$

models/portfolio_pnl.py:135-138

(`_scale_to_target_vol`) applied separately to the smoothed modulated weight ($w = \bar{r}\bar{a}\bar{w}$, using that same day's $\hat{\Sigma}_t$) and to the raw risk-parity weight ($w = w_{\text{rp}}$) for the baseline. `target_vol` (default 10% annualized) is a MODEL PROPERTY, set at training time and persisted in the checkpoint alongside `neutral_band` (§6) – evaluation always scales to what the model was actually meant to be traded at. The $20\times$ cap bounds leverage when the book's own estimated volatility is near zero (e.g. all-neutral signals).

Realized daily PnL is $\text{pnl}(t) = \hat{w}(t)^\top r_{t+1}$ (or, per asset, $\hat{w}_i(t) r_{i,t+1}$ – these sum, per day, to the whole-book figure). Both the modulated book and the risk-parity baseline are reported for train/validation/test, plus a per-calendar-year Sharpe breakdown (§13).

12 Optional Complementary Objective: Training on Portfolio Sharpe

Optional component (opt-in, default off). `sharpe_weight` (default 0) adds a SECOND, COMPLEMENTARY training phase after the per-asset NLL+BCE phase (§8) each epoch. 0 is a byte-identical no-op (verified: `smoke_test.py` test 10a compares a call WITH the new keyword arguments, all disabled, against the pre-existing call signature – identical weights to 10^{-7} tolerance).

12.1 Why this cannot be per-asset

The target-volatility denominator in Eq. 25, $w(t)^\top \hat{\Sigma}_t w(t)$, mixes EVERY asset's current signal together – asset i 's scaled position genuinely depends on what asset j is currently predicting. Unlike Gaussian NLL/BCE, this objective cannot be decomposed into N independent per-asset losses. Two design readings were considered:

	Reading A: scale baseline first	Reading B: scale the modulated weight (adopted)
Formula	$\text{position}_i(t) = \text{const}_i(t) \times s_i(t)$, const precomputed from data alone	Exactly Eq. 25 – matches evaluation exactly
Separable per asset?	Yes – drops into the existing independent loop with zero architectural change	No – requires a joint forward pass and one shared backward()
Fidelity to eval. strategy	Approximation (ignores how the signal itself affects realized book vol)	Exact match: this is a genuine differentiable counterpart of §11

Reading B was the explicit design choice: the training objective should be a faithful differentiable counterpart of the ACTUAL deployed strategy, not a similar-looking simplification.

12.2 The joint training phase

```

1 if track_sharpe:
2     for opt in optimizers: opt.zero_grad()
3     sharpe_loss = _portfolio_sharpe_loss(model, X_train, next_returns_train,
4                                         rp_weights_train, cov_train, direction_horizon,
5                                         sharpe_window, target_vol)
6     (sharpe_weight * sharpe_loss).backward()
7     for i in range(n_assets):
8         _assert_finite_grad(model.assets[i].parameters(), f"epoch {epoch}, asset {i}, Sharpe
9                             phase")
10    for opt in optimizers: opt.step()

```

Listing 16: Phase B - one shared backward() across every asset's parameters.

models/portfolio_lstm.py:1481-1497

```

1 def _portfolio_sharpe_loss(model, X, next_returns, rp_weights, cov, direction_horizon,
2                             sharpe_window, target_vol):
3     mu, sigma = model(X) # EVERY asset, one forward pass
4     signal = (probit(mu[:, -1, :] / sigma[:, -1, :].clamp_min(1e-8)) - 0.5) * 2.0
5     raw_modulated = torch.nan_to_num(rp_weights, nan=0.0) * signal # zero BEFORE arithmetic -
6     # see fixnote
7     w_mod = _trailing_moving_average_torch(raw_modulated, direction_horizon)
8     port_var = torch.einsum("ti,tij,tj->t", w_mod, torch.nan_to_num(cov, nan=0.0), w_mod)
9     scale = torch.clamp((target_vol/252**0.5) / torch.sqrt(port_var.clamp_min(1e-16)), max
10                        =20.0)
11     daily_pnl = (w_mod * scale.unsqueeze(-1) * next_returns).sum(dim=-1)
12     rolling_sharpe = _rolling_sharpe_torch(daily_pnl, sharpe_window) # every position, trailing
13     # window
14     return -rolling_sharpe.mean() # AVERAGED rolling Sharpe

```

Listing 17: The differentiable, JOINT negative averaged rolling Sharpe.

models/portfolio_lstm.py:1182-1276

rp_weights/cov are the PRECOMPUTED, DATA-ONLY constants from §11.1 (computed once before training even starts, reused across every seed/restart) – no gradient flows through them, only through each asset's own probability signal. A single backward() on this joint scalar correctly populates gradients in EVERY asset's parameters via the shared computational graph; each asset's own optimizer then steps using only its own accumulated gradient and its own Adam state – what

is joint is how the gradient was COMPUTED, not the optimizer state that consumes it, so this does not reintroduce weight sharing, only a shared loss term.

Implementation note. A NaN-propagation trap, found and fixed. $\text{NaN} \times 0 = \text{NaN}$ in IEEE754. Rows of `rp_weights/cov` without enough trailing covariance history are NaN (§11.1); `nan_to_num` zeroes them BEFORE any arithmetic (both the weight AND the covariance), not after – had zeroing happened only on the final output, a NaN entry surviving through even one multiplication would poison the entire batch’s gradient. Verified directly (`smoke_test.py` test 10d): a short synthetic series with a real NaN warm-up region produces a finite loss and finite gradients.

$$\text{Sharpe}_\tau = \frac{\overline{\text{pnl}}_{[\tau-W_s+1, \tau]}}{\text{std}(\text{pnl})_{[\tau-W_s+1, \tau]}}, \quad \mathcal{L}_{\text{Sharpe}} = -\frac{1}{T - W_s + 1} \sum_{\tau=W_s-1}^{T-1} \text{Sharpe}_\tau \quad (26)$$

An AVERAGED rolling Sharpe over the whole training period (`sharpe_window` W_s , default 20 days), not a single end-of-period value – the objective reflects consistency across the training window rather than wherever training happened to end.

12.3 Validation selection uses the same composite objective

When tracked, the SAME joint Sharpe is computed on validation (no backward – pure diagnostic/selection signal) and SUBTRACTED from each asset’s own `val_loss_i` before the best-checkpoint comparison:

$$\text{combined_val_loss}_i = \text{val_loss}_i - \lambda_{\text{sharpe}} \cdot \text{val_sharpe} \quad (27)$$

i.e. checkpoint selection uses the EXACT composite objective training optimizes, rather than switching to an isolated Sharpe-only metric – validation-period Sharpe alone, over a few hundred days, is a high-variance statistic; selecting on it in isolation risks picking whichever epoch’s realized path happened to align with the validation window rather than a genuinely better model.

Rationale. A degenerate-solution risk, disclosed, not hidden. Directly optimizing Sharpe on noisy daily FX returns is a documented way for a model to drift toward low-variance, low-conviction bets that inflate mean/std rather than genuine predictive signal – particularly since the signal is bounded to $[-1, 1]$ by construction, which contains but does not eliminate this risk. `sharpe_weight` is deliberately framed as **complementary** to, never a replacement for, the calibrated NLL+BCE objective – the UI explicitly recommends keeping it modest relative to `bce_weight`.

12.4 Empirical verification of the cross-asset coupling

The whole point of Reading B is that asset i ’s training now genuinely depends on asset j ’s current output – the OPPOSITE of §8’s independence guarantee, which only holds when `sharpe_weight=0`. Verified directly (`smoke_test.py` test 10b): the same solo-vs-paired harness used to prove independence (§8) is re-run with `sharpe_weight=1.0` – asset A solo reports hit rate 0.700; the SAME asset A trained alongside an unrelated noise asset B now reports 0.484 – a real, substantial difference, confirming the joint phase’s gradient genuinely reaches across assets (test 10c additionally confirms the objective’s SIGN is correct: training Sharpe improves from -0.10 to 0.65 on a planted profitable signal).

13 Results

13.1 Asset selection

Three pairs were chosen for genuine cross-sectional diversification rather than three similarly-behaving USD crosses: **EURUSD** (the deepest, most liquid pair – Eurozone monetary policy

and broad USD strength/weakness), **USDJPY** (the classic carry/safe-haven pair – JPY tends to strengthen in risk-off episodes precisely when higher-yielding currencies weaken, and is unusually rate-differential-sensitive, making the optional "carry" feature, §3.2, particularly relevant), and **AUDUSD** (a commodity-bloc, risk-on/risk-off currency whose drivers – Chinese demand, commodity prices, the RBA's own cycle – are structurally different from either of the other two). This trio deliberately avoids the redundancy of e.g. EURUSD+GBPUSD (both European legs against USD, highly correlated), while still keeping the experiment small enough to run and report exhaustively.

13.2 Model modes

Three progressively richer configurations were trained on the SAME three pairs, SAME look-back/horizon/split, single seed, single restart – an illustrative comparison of this system's own optional components, **not** a rigorously validated claim of superiority (discussed further below):

Mode	Configuration	Isolates
A: Baseline	<code>features=[log_return,vol,skew,kurt]</code> , <code>sharpe_weight=0</code>	Pure distributional (NLL+BCE) training, minimal features
B: Enriched features	+ <code>carry</code> , <code>cma=[[10,50]]</code> , <code>bandpass=[[10,50]]</code> , still <code>sharpe_weight=0</code>	The effect of richer per-asset input features alone
C: PnL-complementary	Same features as B, <code>sharpe_weight=1.0</code>	Adds the joint portfolio-Sharpe training phase (§12) on top of B

Common hyperparameters: `lookback=30`, `years=8`, `train_frac=0.8`, `test_frac=0.1`, `direction_horizon=5`, `hidden_size=16`, `num_layers=1`, `n_attn_heads=4`, `epochs=300`, `lr=10-3`, `target_vol=0.10`, `device="cpu"` (avoiding the MPS issue of §8.3 entirely for this run), `cross_pairs={}` (every pair fully independent). Training took 109s/112s/204s respectively (mode C's joint Sharpe phase roughly doubles per-epoch cost, as expected – two full phases instead of one).

13.3 Directional hit rate

Mode	Train			Validation			Test		
	EURUSD	USDJPY	AUDUSD	EURUSD	USDJPY	AUDUSD	EURUSD	USDJPY	AUDUSD
A: Baseline	0.530	0.612	0.608	0.392	0.514	0.627	0.491	0.542	0.515
B: Enriched	0.595	0.631	0.628	0.340	0.530	0.506	0.667	0.520	0.519
C: PnL-mode	0.672	0.668	0.680	0.437	0.502	0.530	0.611	0.730	0.541

Train hit rate increases monotonically with model capacity/objective richness (expected – more features and an additional training signal both give the optimizer more to fit). Validation is noticeably noisier and, for EURUSD, WORSE for the richer modes – a single 375-day validation window is a small sample, and this is a single seed/restart, so this should be read as "consistent with normal seed-to-seed variance on a small validation set", not "richer features hurt EURUSD specifically". Test hit rate is mixed across pairs but the richer modes are never uniformly worse than the baseline on test.

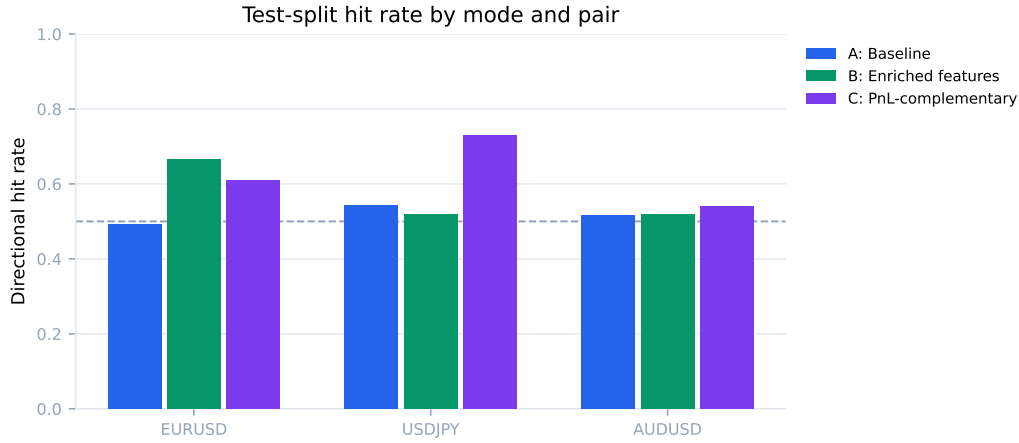


Figure 3: Test-split directional hit rate, all three modes, all three pairs. Dashed line: random chance (50%).

13.4 Confusion matrix (mode C, test split)

The full confusion-matrix breakdown for the richest mode, as a representative example of what every mode/split combination reports:

Pair	TP	FP	TN	FN	Abst.	Coverage	Precision	Recall	F1
EURUSD	49	30	9	7	107	47.0%	62.0%	87.5%	0.726
USDJPY	40	6	6	11	139	31.2%	87.0%	78.4%	0.825
AUDUSD	58	43	14	18	69	65.8%	57.4%	76.3%	0.655

USDJPY combines the highest precision (87.0%) with the lowest coverage (31.2%) – the neutral band (§10) is abstaining on the majority of USDJPY days and speaking only when confident, exactly its intended behavior; AUDUSD’s higher coverage (65.8%) at lower precision reflects the opposite trade-off.

13.5 Portfolio Sharpe: modulated vs. risk-parity baseline

Mode	Train		Validation		Test	
	Modulated	Risk parity	Modulated	Risk parity	Modulated	Risk parity
A: Baseline	1.701	0.040	0.125	-0.359	-0.153	1.764
B: Enriched	2.399	0.040	-1.023	-0.359	0.901	1.764
C: PnL-mode	4.321	0.040	-0.069	-0.359	1.842	1.764

(Annualized Sharpe of daily whole-book PnL; risk-parity baseline is identical across modes within a split by construction – Eq. 22 depends only on realized returns, never on any model’s probabilities.) Train Sharpe rises sharply with mode, most dramatically for C, whose training objective directly targets exactly this quantity (§12) – expected, and not by itself evidence of a better model (this is precisely the in-sample number a Sharpe-trained model is supposed to be able to inflate). The more informative read is TEST, held out from every training and selection decision: modulated Sharpe improves monotonically across modes ($-0.153 \rightarrow 0.901 \rightarrow 1.842$), though it never overtakes the risk-parity baseline’s own test Sharpe (1.764) in modes A/B, and only modestly exceeds it in mode C – on this single train/val/test partition and single seed, richer features plus the complementary

Sharpe objective move the modulated book from clearly worse than the naive diversified baseline to roughly on par with it, not to a decisive outperformance.

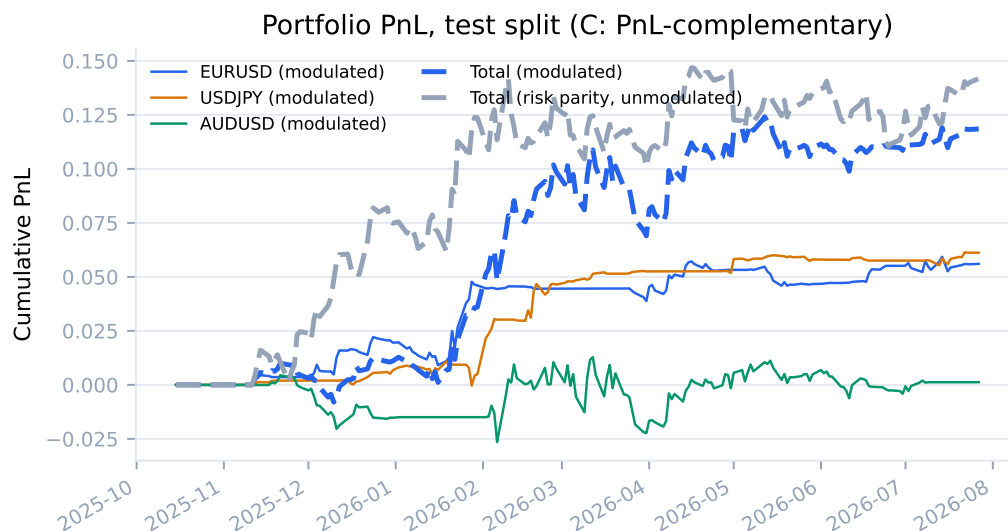


Figure 4: Mode C, test split: cumulative PnL per asset (modulated) and the whole-book total, modulated versus the unmodulated risk-parity baseline.

Year	Days	Sharpe (modulated)	Sharpe (risk parity)
2025	55	1.77	4.72
2026	147	1.99	1.06

Mode C, test split, per-calendar-year breakdown (the test window spans a partial 2025 and a partial 2026) – exactly the per-year Sharpe table now surfaced live in both the Training and Evaluation pages. Neither year is a single outlier driving the whole-period number in either direction.

13.6 Colored cumulative return and predicted probability (mode C, test, EURUSD)

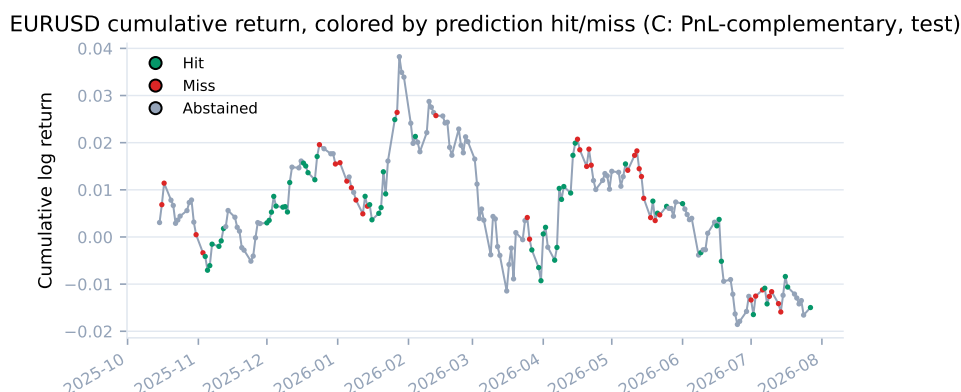


Figure 5: EURUSD's own cumulative (single-day-ahead) log return, test split, mode C – each day colored by whether that day's directional call was a hit, a miss, or an abstention (inside the neutral band).

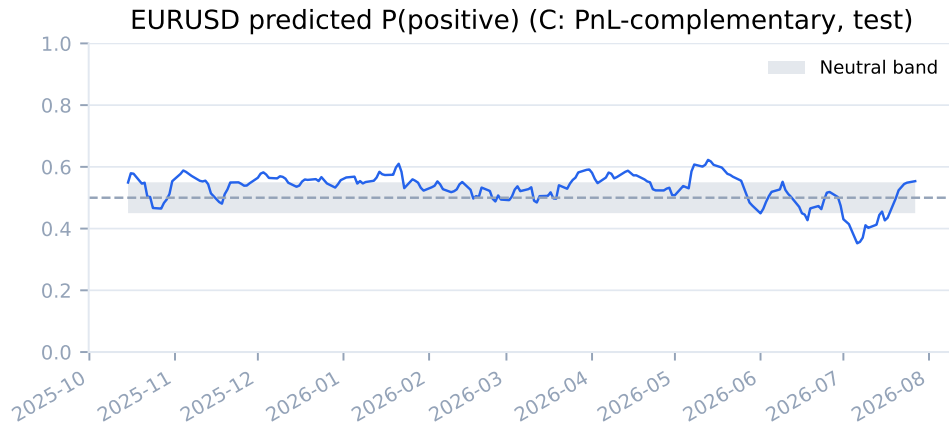


Figure 6: EURUSD's predicted $P(\text{positive})$, test split, mode C. Shaded band: the neutral band (± 0.05 around 0.5) – probabilities inside it are abstentions, not weak directional calls.

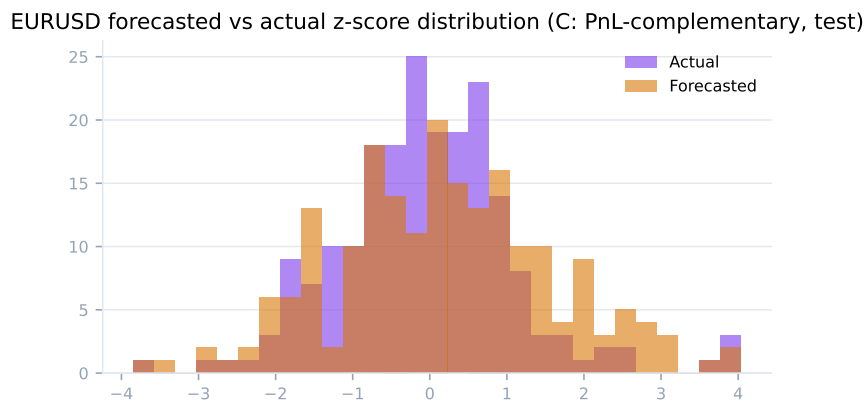


Figure 7: EURUSD forecasted (one sample per row from the model's own $N(\mu, \sigma \hat{\sigma}_{\text{hat}})$) versus actual realized z-score, test split, mode C – a distributional calibration check, not a point-accuracy one.

13.7 Honest limitations of this results section

Rationale. This is a **single seed, single restart, single train/val/test partition**, on three pairs, over one 8-year history ending at the time of writing – enough to demonstrate that every architectural piece described in this document runs correctly end-to-end and produces internally consistent numbers (e.g. hit rate and confusion-matrix coverage/precision agree exactly; the risk-parity baseline is identical across modes as it must be), but **not** enough to claim mode C is statistically better than mode A on this asset universe. A rigorous claim would need multiple seeds (`n_seeds` > 1, this system's own built-in mechanism for exactly this), multiple train/val/test partitions (walk-forward), and ideally more than three assets. The validation-split reversal for EURUSD between modes A and B is a concrete illustration of exactly why: a few hundred validation days is not enough to distinguish "this configuration is worse" from "this seed's validation window happened to be unfavorable for this configuration".

14 Configuration Reference

Every key below lives in `DEFAULT_CONFIG` (`models/portfolio_lstm.py:1643-1767`); a JSON config passed to `main.py` only needs to specify keys that differ from these defaults. "pairs" is the only key with no default.

Key	Default	Meaning
<code>pairs</code>	<i>required</i>	FX pairs, e.g. ["EURUSD", "USDJPY", "AUDUSD"]
<code>lookback</code>	30	Days per input window (L)
<code>years</code>	8	Years of history fetched
<code>train_frac</code> <code>test_frac</code>	/ 0.8 / 0.1	Chronological split fractions (§4.3)
<code>direction_horizon</code>	5	Forward days the z-score label looks (H_d)
<code>rolling_stats_window</code>	20	Trailing window for vol/skew/kurt + label normalization (W_r)
<code>features</code>	[log_return, vol, skew, kurt]	Selected input channels (§3.2)
<code>cma_windows</code> <code>bandpass_windows</code>	/ [] / []	(short, long) window pairs for "cma"/"bandpass"
<code>bandpass_order</code>	3	Butterworth filter order (Eq. 8)
<code>cross_pairs</code>	{}	Per-pair opt-in cross-asset input widening (§6.1)
<code>hidden_size</code>	16	AssetLSTM hidden width H
<code>num_layers</code>	1	LSTM layers (see §8.3 before raising with <code>device="mps"</code>)
<code>dropout</code>	0.1	Dropout, LSTM + attention
<code>n_attn_heads</code>	4	Causal self-attention heads K
<code>epochs</code>	300	Training epochs
<code>lr / weight_decay</code>	10^{-3} / 10^{-4}	Adam hyperparameters (per-asset instance)
<code>bce_weight</code>	1.0	λ_{bce} (Eq. 17); accepts a list to sweep
<code>sharpe_weight</code>	0.0	Optional joint Sharpe phase weight (§12); 0 disables
<code>sharpe_window</code>	20	Rolling window W_s for Eq. 26
<code>neutral_band</code>	0.05	Abstention half-width (§10)
<code>target_vol</code>	0.10	Annualized target volatility (Eq. 25)
<code>n_seeds</code>	1	Independent restarts, best validation log-loss wins
<code>device</code>	"auto"	"auto"/"cpu"/"mps"/"cuda"
<code>save_db</code>	False	Also persist to <code>quant.model_registry</code>

15 Summary of Architectural Decisions

Decision	What it replaced / alternative	Why
Fully independent per-asset training	One shared optimizer over every asset's parameters	Isolates each asset's own gradient signal; verified byte-identical solo-vs-paired (§8)
<code>cross_pairs</code> , default off	Unconditional full cross-asset mixing	Opt-in, per-pair, explicit – no way to dial back an always-on design
Causal self-attention over LSTM output	LSTM's final hidden state alone	Avoids the fixed-size bottleneck of $h_n[-1]$ for longer lookbacks; verified causal via perturbation test

Decision	What it replaced / alternative	Why
Gaussian NLL + direction BCE	Plain regression (MSE/Huber)	Anti-collapse: BCE forces μ to spread per-sample sign, not settle on the unconditional mean
Neutral band as pure postprocessing	Baked into training/reporting	Enables live, client-side band recomputation with no retraining
Risk parity, not equal weight	Naive $1/N$ weighting	Accounts for correlation in the risk contribution, not just variance
Smooth product, not signal, before vol-scaling	Smoothing the raw signal alone	Matches the actual deployed strategy exactly (Reading B, §12)
Butterworth band-pass via <code>lfilter</code>	<code>filtfilt</code> (the "usual" way)	<code>filtfilt</code> is non-causal – leaks future returns
Precomputed risk parity for the Sharpe objective	Solving ERC inside the training loop	The ERC solve isn't differentiable (external SLSQP call); it's also DATA-ONLY, so precomputing once is exact, not an approximation
<code>_assert_finite_grad</code> safety net	Silent NaN propagation	Turns a wasted 300-epoch run into an immediate, actionable error